

Week 3: Advanced Security and Final Reporting

1. Basic Penetration Testing

Nmap — Network Scanning

Run basic scans to identify open ports and running services on the target host:

```
# Scan open ports and detect service versions
nmap -sV 10.10.111.246
```

```
# Scan all ports
nmap -p- 10.10.111.246
```

```
# Check for known vulnerabilities
nmap --script vuln 10.10.111.246
```

Browser-Based Testing

Test the application manually in the browser for common vulnerabilities:

- LFI — try path traversal in URL parameters:

```
GET /help?page=../../../../etc/passwd
```

- XSS — inject a script tag in any input field:

```
<script>alert(document.cookie)</script>
```

- Brute-Force — use Burp Suite Intruder or a curl loop:

```
for i in {1..10}; do
```

```
  curl -X POST /login -d 'username=admin&password=guess'$i
done
```

2. Set Up Basic Logging

Use the winston library to log security events to the console and to a file.

Logger Setup

```
const winston = require('winston');

const logger = winston.createLogger({
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'security.log' })
  ]
});

logger.info('Application started');
```

Log Login Attempts

Add logging to the login route using the existing parameterised query:

```
pool.query('SELECT * FROM admin WHERE username = ?', [username],
  async function(err, rows) {
    if (err || rows.length === 0) {
      logger.warn(`Failed login attempt for user: ${username}`);
      return res.status(400).send('User not found');
    }
    logger.info(`Login attempt by ${username} at ${new Date().toISOString()}`);
  }
);
```

Log Security Events

Add a log line at every important security decision point:

```
// Failed input validation
logger.warn(`Invalid input on ${req.path} from ${req.ip}`);

// File upload rejected
logger.warn(`Rejected upload: ${file.originalname} from ${req.ip}`);

// Rate limit triggered
logger.warn(`Rate limit hit on /login from ${req.ip}`);
```

3. Create a Simple Checklist

Best practices to verify before deploying the application to production:

Validate All Inputs

- Use the validator library on every endpoint that accepts user data.
 - `isEmail()`, `isAlphanumeric()`, `isStrongPassword()`, `escape()`
- Reject requests that fail validation before they reach business logic.

Hash and Salt Passwords

- Use `bcrypt` with cost factor 12 — never store plain text.

```
const bcrypt = require('bcrypt');
```

```
const hash = await bcrypt.hash(password, 12);
```

```
const match = await bcrypt.compare(password, hash);
```

Use HTTPS for Data Transmission

Create an HTTPS server using a TLS certificate:

```
const https = require('https');
```

```
const fs = require('fs');
```

```
const httpsOptions = {  
  key: fs.readFileSync('key.pem'),  
  cert: fs.readFileSync('cert.pem'),  
};
```

```
const httpsServer = https.createServer(httpsOptions, app);  
httpsServer.listen(8443, () => {  
  console.log('HTTPS Server listening on port 8443');  
});
```